Title:

POLYMER

Effort Analysis Agenda

Author:

Mohammad-Sajad Kasaei

University of Isfahan

smskasaei@gmail.com

1. Welcome and Introduction

Thank you for participating in this study. Your contribution is invaluable for understanding the effort required to define model transformations and comparisons using different languages. This study aims to analyze the effort of our proposed N-way Merging Language (NML) by comparing it against established Epsilon languages (ECL, EML, ETL).

Your task is to write rules for a given domain model based on a UML meta-model. You will be asked to perform these tasks using different languages and in various scenarios. Please record the time you spend on each task as accurately as possible.

Thank you once again for your participation. Please ask any questions you may have before you begin the tasks.

2. Evaluation Objective

The primary objective of this evaluation is to assess and compare the effort required by developers to define model transformations and merging rules using NML, specifically its Octopus, Ours, and Transfer rule types across N-way merging scenarios (2, 4, and 7 models). This will be benchmarked against the effort required using the Epsilon Comparison Language (ECL), Epsilon Merging Language (EML), and Epsilon Transformation Language (ETL).

3. Study Design

Participants: You, as the participant.

Materials:

- A Domain Model for the SDG (Service Delivery Gateway) domain, representing a pickup and delivery system.
- A UML meta-model defining the concepts of Model, Package, Class, Attribute, and Association.
- The NML language syntax and examples for Octopus, Ours, and Transfer rules.
- The syntax and examples for ECL, EML, and ETL.

Tasks: You will be asked to write specific rules that apply to the provided SDG domain model using the different languages. These tasks are designed to cover various aspects of model manipulation and comparison.

Procedure:

1. You will be presented with the SDG domain model and the UML meta-model.
2. You will then be instructed to write a series of rules using one of the specified languages (NML, ECL, EML, ETL).
3. The rule-writing tasks will be organized into different scenarios based on the number of models involved (2, 4, or 7) and whether a base version is provided.
4. For each rule you write, you are instructed to record the time taken in the provided tables.
5. After completing the rule-writing tasks for a language or scenario, you will answer a short questionnaire.

4. Detailed Tasks and Scenarios

You will be performing the following tasks. Please pay close attention to the language, rule type (for NML), and scenario for each task.

A. Comparison and Merging Language Tasks:

You will write rules using the following languages. These tasks are designed to simulate the equivalent operations for comparison and transformation.

- Epsilon Comparison Language (ECL):
  - Write comparison rules to identify differences or commonalities within the model set (analogous to Octopus/Ours).
  - Element must be included in the rules: For each scenario, you will write ECL rules for Model, Package, Class, Attribute, Association.
  - Record time for each element type separately.
- Epsilon Merging Language (EML):
  - Write merging rules to combine models (primarily for 2-way merging, but conceptually extended).
  - Element must be included in the rules: Model, Package, Class, Attribute, Association.
  - Record time for each element type separately.
- Epsilon Transformation Language (ETL):
  - Write transformation rules to map elements from one model or set of models to another.
  - Element must be included in the rules: Model, Package, Class, Attribute, Association.
  - Record time for each element type separately.

B. NML Tasks:

You will write rules using NML in the following rule types and scenarios:

- NML Rule Type: Octopus
  - Definition: Rules that apply to elements present in all model versions.
  - Scenarios:
    - For Merging 2 Models (No Base Version Provided)
    - For Merging 4 Models (No Base Version Provided)
    - For Merging 7 Models (No Base Version Provided)
    - For Merging 2 Models (With Base Version Provided)
    - For Merging 4 Models (With Base Version Provided)
    - For Merging 7 Models (With Base Version Provided)
  - Element must be included in the rules: For each scenario, you will write Octopus rules for Model, Package, Class, Attribute, and Association elements.
  - Record time for each element type separately.
- NML Rule Type: Ours
  - Definition: Rules that apply to elements present in some (but not necessarily all) model versions.
  - Scenarios:

        - For Merging 2 Models (No Base Version Provided)
        - For Merging 4 Models (No Base Version Provided)
        - For Merging 7 Models (No Base Version Provided)
        - For Merging 2 Models (With Base Version Provided)
        - For Merging 4 Models (With Base Version Provided)
        - For Merging 7 Models (With Base Version Provided)
    - Element must be included in the rules: For each scenario, you will write Ours rules for Model, Package, Class, Attribute, and Association elements.
    - Record time for each element type separately.
- NML Rule Type: Transfer
    - Definition: Rules that handle elements unique to specific versions or describe the transfer of elements.
    - Scenarios:
        - For Merging 2 Models
        - For Merging 4 Models
        - For Merging 7 Models
    - Element must be included in the rules: For each scenario, you will write Transfer rules for Model, Package, Class, Attribute, and Association elements.
    - Record time for each element type separately.

5. Evaluation Metrics

We are collecting two types of data:

A. Quantitative Metrics:
   - Rule Writing Time: The primary metric. You will record the time (in minutes and seconds) spent writing each specific rule instance for each element type, language, and scenario combination.
   - Details Collected:
     - Language Used (NML, ECL, EML, ETL)
     - NML Rule Type (Octopus, Ours, Transfer)
     - Scenario (Number of models, With/Without Base Version)
     - Element Type (Model, Package, Class, Attribute, Association)
     - Time Taken (e.g., MM:SS)

B. Qualitative Metrics:
   - Questionnaire Responses: After completing tasks, you will answer questions about your experience. These questions will cover aspects such as:
     - Ease of understanding the language and its constructs.
     - Perceived complexity of the tasks.
     - Confidence in the correctness of the rules written.
     - Comparison of NML's expressiveness and efficiency to ECL, EML, and ETL.
     - Overall satisfaction with the language.

6. Data Recording

Please use the following tables to meticulously record the time spent on each rule-writing task. Ensure you fill in all relevant details.

1. Table 1: NML - Octopus Rule Type
   - Rule Type: Octopus (Elements present in all model versions)

Element Type	Scenario: 2 Models (No Base)	Scenario: 4 Models (No Base)	Scenario: 7 Models (No Base)	Scenario: 2 Models (With Base)	Scenario: 4 Models (With Base)	Scenario: 7 Models (With Base)
Model						
Package						
Class						
Attribute						
Association						

2. Table 2: NML - Ours Rule Type
   - Rule Type: Ours (Elements present in some model versions)

Element Type	Scenario: 2 Models (No Base)	Scenario: 4 Models (No Base)	Scenario: 7 Models (No Base)	Scenario: 2 Models (With Base)	Scenario: 4 Models (With Base)	Scenario: 7 Models (With Base)
Model						
Package						
Class						
Attribute						
Association						

3. Table 3: NML - Transfer Rule Type
   - Rule Type: Transfer (Elements unique to specific versions)

Element Type	Scenario: 2 Models	Scenario: 4 Models	Scenario: 7 Models
Model			
Package			
Class			
Attribute			
Association			

4. Table 4: Epsilon Comparison Language (ECL)
   - Rule Type: Comparison

Element Type	Time to Write Rule(s)
Model	
Package	
Class	
Attribute	
Association	

5. Table 5: Epsilon Merging Language (EML)
   - Rule Type: Merge (2-way)

Element Type	Time to Write Rule(s)
Model	
Package	
Class	
Attribute	
Association	

6. Table 6: Epsilon Transformation Language (ETL)
   - Rule Type: Transform

Element Type	Time to Write Rule(s)
Model	
Package	
Class	
Attribute	
Association	

Appendix A. Epsilon Comparison Language (ECL)

Syntax:

```
rule <name>
        match<leftParameterName>:<leftParameterType>
        with <rightParameterName>:<rightParameterType>
        (extends <ruleName>(,<ruleName>)*)?{
                (guard (:expression )|({statementBlock}))?
                Compare (:expression )|({statementBlock})
                (do {statementBlock})?
        }
```

Example:

```
rule CompareMyClass
        match l : LeftVersion!Class
        with r : RightVersion!Class
                {
                compare : l.name= r.name
        }
```

Appendix B. Epsilon Merging Language (EML)

Syntax:

```
rule <name>
        merge <leftParameterName>:<leftParameterType>
        with <rightParameterName>:<rightParameterType>
        into (<targetParameterName>:<targetParameterType>,
                (<targetParameterName>:<targetParameterType>)*)?
        (extends <ruleName>(,<ruleName>)*)?{
                (do {statementBlock})?
        }
```

Example:

```
rule MergeMyClass
        merge l : LeftVersion!Class
        with r : RightVersion!Class
        into t : TargetVersion!Class
                {
                t.name= r.name;
                t.package = r.package.equivalent();
        }
```

Appendix C. Epsilon Transformation Language (ETL)

Syntax:

```
rule <name>
        transform <parameterName>:<sourceName>!<parameterType>
        to (<targetParameterName>:<targetParameterType>,
                (<targetParameterName>:<targetParameterType>)*)?
        (extends <ruleName>(,<ruleName>)*)?{
                (do {statementBlock})?
        }
```

Example:

```
rule TransformMyClass
        transform vs : SourceVersion!Class
        to t : TargetVersion!Class
                {
                t.name= vs.name
                t.package = vs.package.equivalent();
        }
```

Appendix D. N-way Merging Language (NML): Octopus

Syntax:

```
rule <name>
        mergeOctopus <ParameterName>:<ParameterType>
        with < ParameterName >:< ParameterType >
                (<ParameterName >:< ParameterType >)*)?
        (withBase < ParameterName >:< ParameterType >)?
        into (<targetParameterName>:<targetParameterType>,
                (<targetParameterName>:<targetParameterType>)*)?
        (extends <ruleName>(,<ruleName>)*)?{
                (do {statementBlock})?
        }
```

Example:

```
rule OctopusMyClass
        mergeOctopus v1 : FirstVersion!Class
        with v2 : SecondVersion!Class,
                v3 : ThirdVersion!Class
        into t : TargetVersion!Class
                {
                t.name= v1.name;
                t.package = v1.package.equivalent();
        }


rule OctopusBaseMyClass
        mergeOctopus v1 : FirstVersion!Class
        with v2 : SecondVersion!Class,
                v3 : ThirdVersion!Class
        withBase v0 : BaseVersion!Class
        into t : TargetVersion!Class
                {
                t.name= v1.name
                t.package = v1.package.equivalent();
        }
```

Appendix F. N-way Merging Language (NML): Ours

Syntax:

```
rule <name>
        mergeOurs <ParameterName>:<ParameterType>
        with < ParameterName >:< ParameterType >
                (<ParameterName >:< ParameterType >)*)?
        (withBase < ParameterName >:< ParameterType >
                (exists_in <NumberOfVersions>)?)?
        priority < PriorityListName >: [<parameterName>(,<parameterName>)*]
                (, <PriorityListName> : [<parameterName>(,<parameterName>)*])*
        into (<targetParameterName>:<targetParameterType>,
                (<targetParameterName>:<targetParameterType>)*)?
        (extends <ruleName>(,<ruleName>)*)?{
                (do {statementBlock})?
        }
```

Example:

```
rule OursMyClass
        mergeOurs v1 : FirstVersion!Class
        with v2 : SecondVersion!Class,
                v3 : ThirdVersion!Class
        priority P1 : [ v1, v2, v3 ]
        into t : TargetVersion!Class
                {
                t.name= v1.name;
                t.package = v1.package.equivalent();
        }


rule OursBaseMyClass
        mergeOurs v1 : FirstVersion!Class
        with v2 : SecondVersion!Class,
                v3 : ThirdVersion!Class
        withBase v0 : BaseVersion!Class
                exists_in 2
        priority P1 : [ v0, v1, v2, v3 ]
        into t : TargetVersion!Class
                {
                t.name= P1.name
                t.package = P1.package.equivalent();
        }
```

Appendix G. N-way Merging Language (NML): Transfer

Syntax:

```
rule <name>
        transfer <parameterName>:<sourceName>!<parameterType>
        from < sourceName >: (<versionName>(, <versionName>)*)
        to (<targetParameterName>:<targetParameterType>,
                (<targetParameterName>:<targetParameterType>)*)?
        (extends <ruleName>(,<ruleName>)*)?{
                (do {statementBlock})?
        }
```

Example:

```
rule TransferMyClass
        transfer vs : SourceVersion!Class
        from SourceVersion : (FirstVersion, SecondVersion, ThirdVersion)
        to t : TargetVersion!Class
                {
                t.name= vs.name
                t.package = vs.package.equivalent();
        }
```